



Politechnika Wrocławska

Sprawozdanie: Struktury Danych - Projekt 3

Danylo Mikhnytksyi 284125, Maksym Mahaz 28115

Grupa 6, 11:15 — 13:00

Informacje ogólne

Przedmiot: Struktury danych
Autorzy: Danylo Mikhnytksyi, Maksym Mahaz
Grupa projektowa: 6 (godziny 11:15 — 13:00)
Repozytorium GitHub: https://github.com/tabbbyzzxc/pwr_sd_pr3

Spis treści

1	Wprowadzenie	3
2	Adresowanie otwarte	3
2.1	Wstęp	3
3	Robin Hood	3
3.1	Wstęp	3
4	Haszowanie kukułcze	4
4.1	Wstęp	4
5	Badania	4
5.1	Badanie wstawiania do tablicy haszującej	4
5.2	Badanie usuwania z tablicy haszującej	7
6	Wnioski	9

1 Wprowadzenie

Tablice haszujące (mieszejące) to struktury danych, które pozwalają na bardzo szybkie wstawianie, wyszukiwanie i usuwanie elementów. Ich działanie opiera się na wykorzystaniu funkcji haszującej, która przekształca klucz elementu na indeks w tablicy. W idealnym przypadku operacje te wykonują się w czasie stałym $O(1)$. Jednakże ze względu na to, że przestrzeń kluczy jest zazwyczaj znacznie większa niż rozmiar tablicy, mogą wystąpić tak zwane kolizje, czyli sytuacje, w których funkcja haszująca przypisze dwóm różnym kluczom ten sam indeks. Istnieje wiele metod rozwiązywania kolizji, z których najpopularniejszymi są metoda łańcuchowa (ang. *chaining*) oraz adresowanie otwarte (ang. *open addressing*). Niniejsze sprawozdanie skupia się na analizie metod opartych o adresowanie otwarte oraz omawia zaawansowane techniki takie jak haszowanie Robin Hooda i haszowanie kukulcze.

2 Adresowanie otwarte

2.1 Wstęp

Adresowanie otwarte to metoda rozwiązywania kolizji polegająca na poszukiwaniu wolnego miejsca w samej tablicy, gdy pod wyliczonym indeksem znajduje się już inny element. Proces poszukiwania nazywany jest próbkowaniem (ang. *probing*). Do najczęstszych wariantów próbkowania należą:

- **Próbkowanie liniowe:** W przypadku kolizji sprawdzana jest kolejna komórka tablicy (z krokiem 1). Główną wadą tej metody jest powstawanie tzw. klastrów pierwotnych (ciągów zajętych komórek), co drastycznie spowalnia działanie algorytmu przy wyższym stopniu wypełnienia tablicy (tzw. *load factor*).
- **Próbkowanie kwadratowe:** Odstęp między kolejnymi sprawdzanymi pozycjami rośnie kwadratowo. Pomaga to wyeliminować problem klastrowania pierwotnego, jednak wciąż może występować klastrowanie wtórne.
- **Haszowanie podwójne:** Do wyznaczenia kroku próbkowania używana jest druga, niezależna funkcja haszująca. Metoda ta oferuje najlepsze rozproszenie elementów i najskuteczniej minimalizuje ryzyko powstawania jakichkolwiek klastrów.

3 Robin Hood

3.1 Wstęp

Haszowanie Robin Hooda (ang. *Robin Hood hashing*) to ulepszenie klasycznego adresowania otwartego (zazwyczaj próbkowania liniowego). Jej główna idea opiera się na zasadzie „zabieraj bogatym, dawaj biednym”. Każdy element w tablicy pamięta swój dystans od oryginalnie wyznaczonego indeksu (tzw. *Probe Sequence Length* - PSL). Podczas wstawiania nowego elementu, jeśli napotkamy zajętą komórkę, porównujemy PSL elementu wstawianego z PSL elementu znajdującego się w komórce. Jeśli wstawiany element ma większy dystans (jest „biedniejszy”), zamienia się miejscami z elementem w komórce. Wypchnięty element kontynuuje poszukiwanie nowego miejsca. Dzięki tej technice znacząco zmniejsza się wariancja czasów wyszukiwania, a maksymalny czas operacji staje się znacznie bardziej przewidywalny i krótszy niż w zwykłym próbkowaniu liniowym.

4 Haszowanie kukułcze

4.1 Wstęp

Haszowanie kukułcze (ang. *Cuckoo hashing*) to metoda, która w odróżnieniu od adresowania otwartego używa co najmniej dwóch różnych tablic (lub jednej tablicy, ale podzielonej) i dwóch niezależnych funkcji haszujących. Każdy klucz ma dokładnie dwa możliwe miejsca, w których może się znajdować. Podczas wstawiania, element jest umieszczany w pierwszej możliwej lokalizacji. Jeśli miejsce to jest zajęte, nowy element „wypycha” starego lokatora (niczym kukułka wyrzucająca jaja z gniazda). Wyrzucony element jest następnie przenoszony do swojej alternatywnej lokalizacji, co może z kolei wypchnąć kolejny element. Taka reakcja łańcuchowa trwa aż do znalezienia pustego miejsca. Główną zaletą haszowania kukułczego jest gwarantowany czas $O(1)$ dla operacji wyszukiwania i usuwania w najgorszym przypadku, ponieważ każdy element trzeba szukać tylko w dwóch konkretnych miejscach.

5 Badania

Poniższe badania skupiają się na analizie wydajności trzech strategii adresowania otwartego: próbkowania liniowego, kwadratowego oraz haszowania podwójnego. Przeprowadzono testy polegające na wstawianiu oraz usuwaniu pakietów danych (o rozmiarach 10, 50, 100, 500, 1000 elementów) z tablicy mieszającej przy jej początkowym wypełnieniu wynoszącym 25%, 50% oraz 75%. Wykresem bazowym jest średnia liczba porównań (prób) potrzebna na wykonanie jednej operacji.

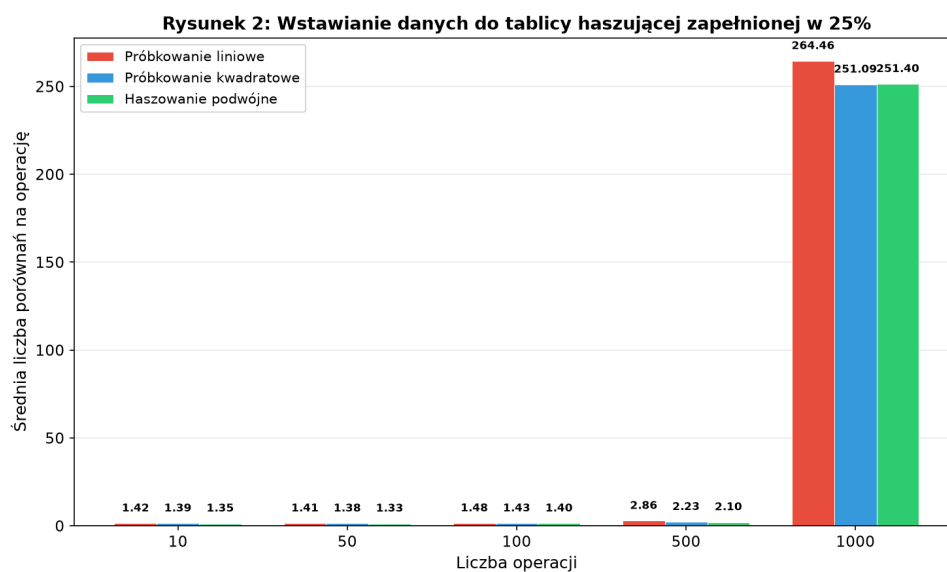
5.1 Badanie wstawiania do tablicy haszującej

Operacja wstawiania (Insert) polegała na dołączaniu nowych kluczy do tablicy. Warto zauważyć, że im wyższy stopień wypełnienia tablicy, tym rośnie średnia liczba porównań.

Rysunek 1: Wyniki wstawiania do tablicy haszującej wypełnionej w 25%

	10	50	100	500	1000
Próbkowanie liniowe	1.416	1.407	1.479	2.857	264.458
Próbkowanie kwadratowe	1.394	1.382	1.43	2.232	251.092
Haszowanie podwójne	1.35	1.326	1.397	2.103	251.395

Rysunek 1: Tablica przedstawiająca wyniki dla wstawiania danych do tablicy haszującej wypełnionej w 25%



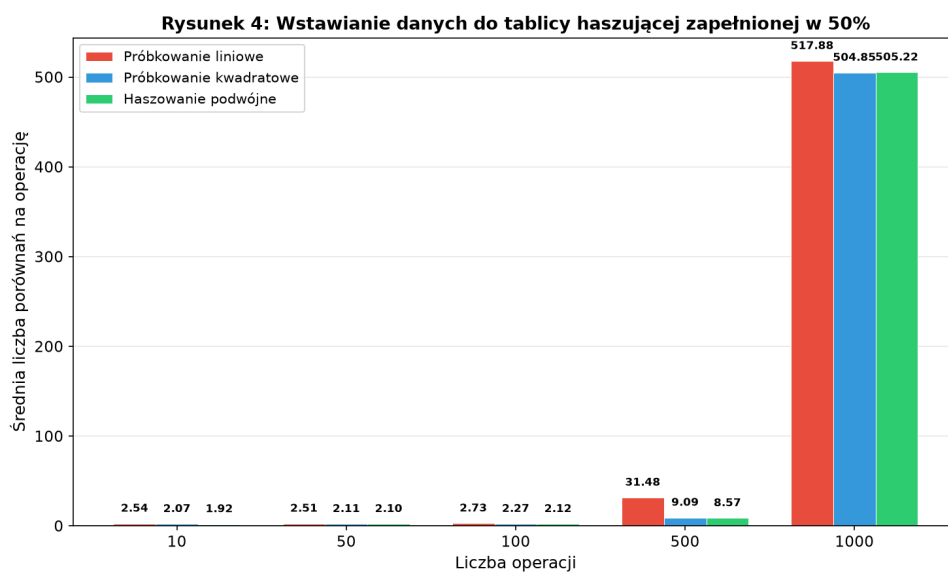
Rysunek 2: Wykres dla wstawiania danych do tablicy haszującej wypełnionej w 25%

Przy wypełnieniu rzędu 25% kolizje występują rzadko. Próbkowanie liniowe, kwadratowe i haszowanie podwójne charakteryzują się bardzo niską liczbą porównań (bliską 1-1.5 na operację). Różnice między algorytmami są na tym etapie niemal niezauważalne, ponieważ tablica posiada mnóstwo wolnych miejsc.

Rysunek 3: Wyniki wstawiania do tablicy haszującej wypełnionej w 50%

	10	50	100	500	1000
Próbkowanie liniowe	2.542	2.506	2.733	31.477	517.881
Próbkowanie kwadratowe	2.074	2.114	2.267	9.092	504.849
Haszowanie podwójne	1.922	2.095	2.119	8.575	505.216

Rysunek 3: Tablica przedstawiająca wyniki dla wstawiania danych do tablicy haszującej wypełnionej w 50%



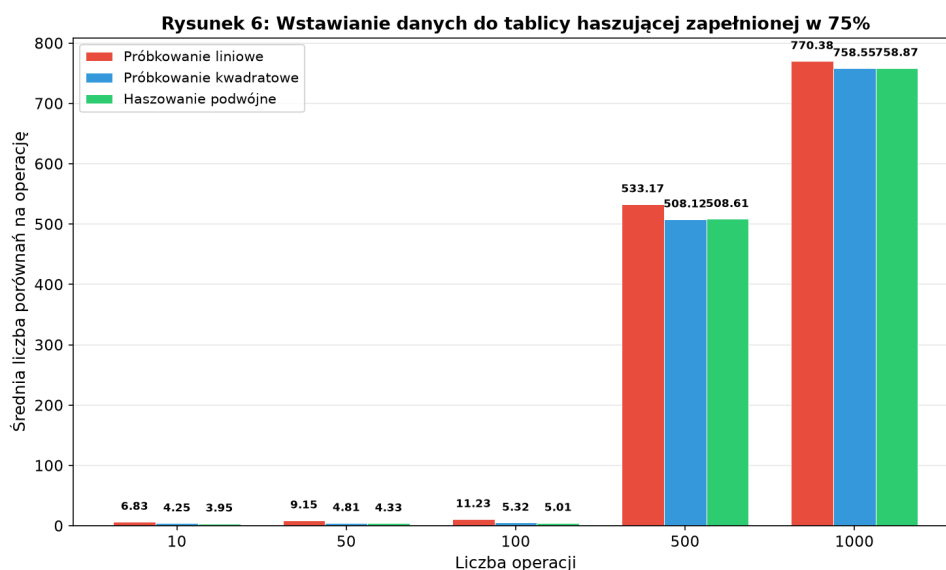
Rysunek 4: Wykres dla wstawiania danych do tablicy haszującej wypełnionej w 50%

Zapełnienie 50% ukazuje pierwsze słabości próbkowania liniowego. Tworzące się tzw. klastry pierwotne zaczynają powoli degradować wydajność, zmuszając algorytm do sprawdzania coraz dłuższych ciągów zajętych komórek. Próbkowanie kwadratowe i haszowanie podwójne utrzymują przewagę i stabilność, skuteczniej omijając zagęszczone obszary.

Rysunek 5: Wyniki wstawiania do tablicy haszującej wypełnionej w 75%

	10	50	100	500	1000
Próbkowanie liniowe	6.828	9.146	11.227	533.166	770.378
Próbkowanie kwadratowe	4.254	4.814	5.316	508.118	758.554
Haszowanie podwójne	3.95	4.327	5.008	508.61	758.872

Rysunek 5: Tablica przedstawiająca wyniki dla wstawiania danych do tablicy haszującej wypełnionej w 75%



Rysunek 6: Wykres dla wstawiania danych do tablicy haszującej wypełnionej w 75%

Gdy zajętość tablicy osiąga 75%, różnice stają się drastyczne. Próbkowanie liniowe odnotowuje olbrzymi skok liczby średnich porównań - dodawanie nowych elementów staje się bardzo kosztowne (wydajność drastycznie spada). Haszowanie podwójne wciąż radzi sobie zdecydowanie najlepiej, a próbkowanie kwadratowe wykazuje umiarkowane pogorszenie wyników ze względu na zjawisko klastrowania wtórnego.

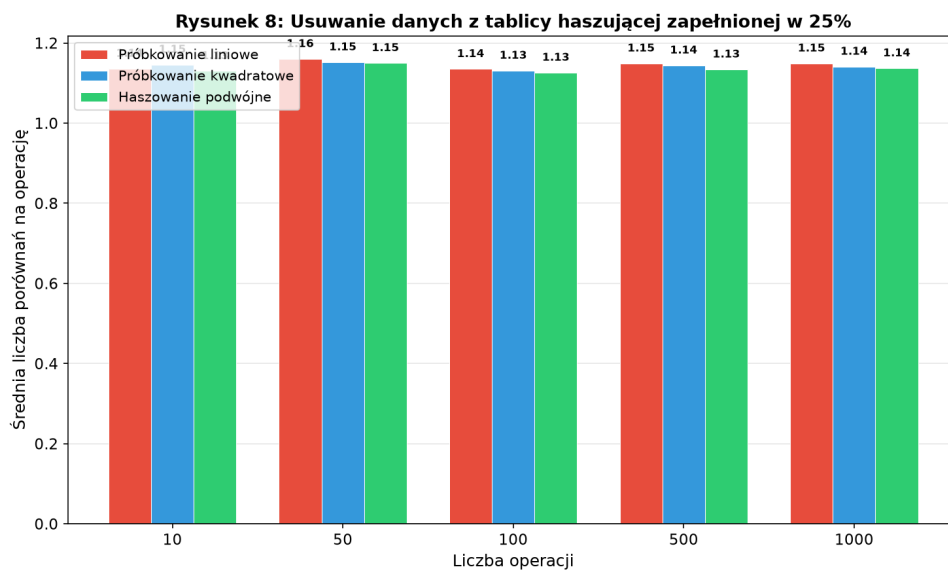
5.2 Badanie usuwania z tablicy haszującej

W przypadku adresowania otwartego usuwanie (Delete) opiera się najczęściej o technikę wstawiania specjalnych znaczników (tzw. "Tombstones" lub węzłów oznaczonych jako USUNIĘTE). W badaniach zliczono ilość próbkowania potrzebną do odnalezienia usuwanego klucza. Zjawiska spadku wydajności są tutaj analogiczne do operacji wstawiania, gdyż do usunięcia elementu niezbędne jest jego uprzednie wyszukanie.

Rysunek 7: Wyniki usuwania z tablicy haszującej wypełnionej w 25%

	10	50	100	500	1000
Próbkowanie liniowe	1.136	1.16	1.135	1.149	1.149
Próbkowanie kwadratowe	1.146	1.152	1.131	1.144	1.141
Haszowanie podwójne	1.13	1.15	1.126	1.134	1.137

Rysunek 7: Tablica przedstawiająca wyniki dla usuwania danych z tablicy haszującej wypełnionej w 25%



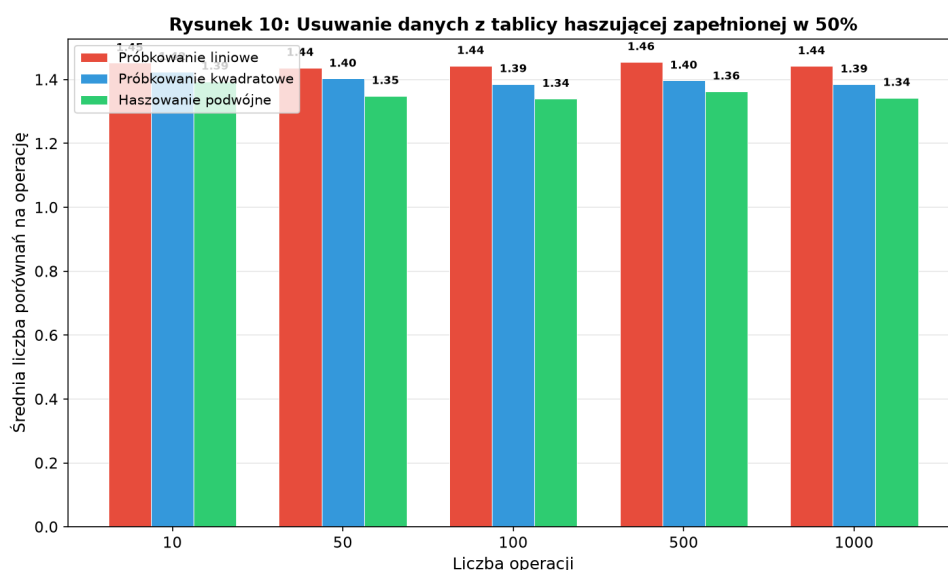
Rysunek 8: Wykres dla usuwania danych z tablicy haszującej wypełnionej w 25%

W luźno wypełnionej tablicy (25%) usunięcie danych wymaga sprawdzenia niewiele więcej niż jednej komórki, niezależnie od zastosowanej metody. Odnalezienie elementu jest błyskawiczne ze względu na brak długich łańcuchów kolizji.

Rysunek 9: Wyniki usuwania z tablicy haszującej wypełnionej w 50%

	10	50	100	500	1000
Próbkowanie liniowe	1.452	1.437	1.443	1.455	1.442
Próbkowanie kwadratowe	1.424	1.403	1.386	1.397	1.385
Haszowanie podwójne	1.394	1.349	1.34	1.363	1.343

Rysunek 9: Tablica przedstawiająca wyniki dla usuwania danych z tablicy haszującej wypełnionej w 50%



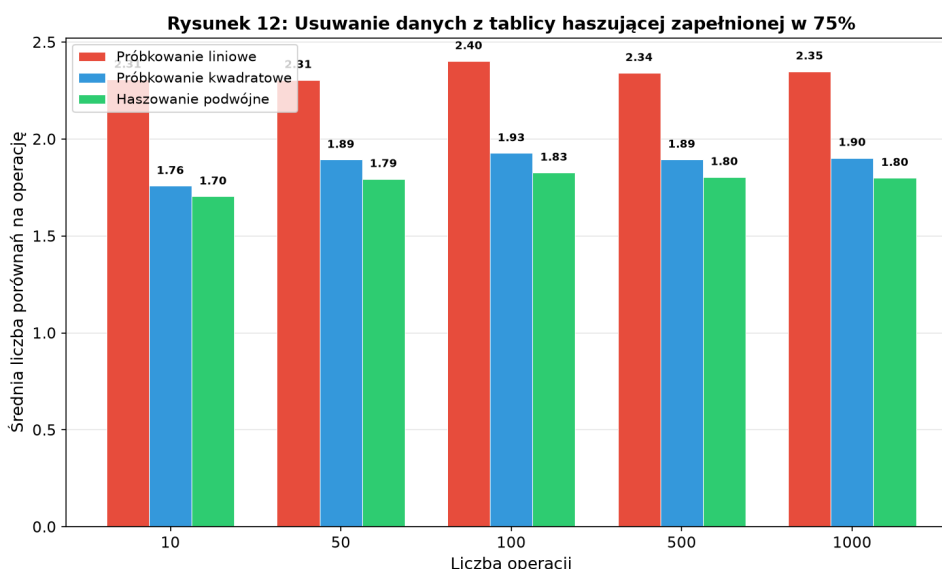
Rysunek 10: Wykres dla usuwania danych z tablicy haszującej wypełnionej w 50%

Przy 50% wypełnieniu proces przeszukiwania dla adresowania liniowego zaczyna spowalniać. Klastry zbudowane podczas wstawiania stają się na tyle długie, że wyszukiwanie elementów wymaga kilkukrotnego odpytania tablicy. Wypadkowo lepiej radzi sobie tu podwójne haszowanie, gdzie rozstrzelanie elementów gwarantuje szybsze dotarcie do szukanego klucza.

Rysunek 11: Wyniki usuwania z tablicy haszującej wypełnionej w 75%

	10	50	100	500	1000
Próbkowanie liniowe	2.306	2.305	2.402	2.341	2.349
Próbkowanie kwadratowe	1.758	1.893	1.927	1.894	1.903
Haszowanie podwójne	1.704	1.794	1.827	1.804	1.8

Rysunek 11: Tablica przedstawiająca wyniki dla usuwania danych z tablicy haszującej wypełnionej w 75%



Rysunek 12: Wykres dla usuwania danych z tablicy haszującej wypełnionej w 75%

Najwyższe zaprezentowane wypełnienie (75%) jednoznacznie potwierdza ułomność próbkowania liniowego. Znalezienie węzła do usunięcia wymaga liniowego przejścia przez spory blok zbitych ze sobą komórek. Próbkowanie kwadratowe wypada znacznie lepiej, natomiast absolutnym zwycięzcą okazuje się haszowanie podwójne, wymagające najmniejszej liczby porównań w celu zlokalizowania odpowiedniego wpisu.

6 Wnioski

Przeprowadzone badania i analiza wybranych metod rozwiązywania kolizji pozwalają na wyciągnięcie następujących wniosków:

1. **Wpływ stopnia wypełnienia (*load factor*):** Stopień wypełnienia tablicy haszującej jest krytycznym parametrem wpływającym na wydajność. O ile dla wartości

rzędu 25% każda z metod radzi sobie doskonale w czasie stałym $O(1)$, o tyle przekroczenie poziomu 50-70% dramatycznie pogarsza charakterystyki czasowe niektórych algorytmów.

2. **Problemy adresowania liniowego:** Próbkowanie liniowe to podejście najprostsze w implementacji, ale bardzo wrażliwe na powstawanie klastrów pierwotnych. Zlepianie się wielu kluczy w ciągle bloki potęguje ilość kolizji dla kolejnych elementów.
3. **Przewaga haszowania podwójnego:** Haszowanie podwójne charakteryzuje się najrówniejszym rozkładem i najniższą degradacją w czasie. Dzięki wykorzystaniu dodatkowej funkcji haszującej do określenia kroku, unika ono zarówno klastrowania pierwotnego, jak i wtórniego, co było wyraźnie widać w próbach wstawiania i usuwania przy 75% wypełnieniu.
4. **Nowoczesne podejścia:** Rozwiązania w stylu *Robin Hood hashing* oraz *Cuckoo hashing* stanowią potężną alternatywę tam, gdzie klasyczne próbkowanie nie wystarcza. Gwarantują one odpowiednio stałe limity najgorszego przypadku lub niwelują wariancję czasów operacji.

W praktyce dobór strategii zarządzania kolizjami zawsze stanowi kompromis pomiędzy stopniem skomplikowania kodu a wymaganą optymalizacją. Tablice z adresowaniem liniowym wymagają bardzo restrykcyjnego powiększania rozmiaru (*rehashing*) dużo wcześniej (często już przy $\approx 50\%$ wypełnienia), podczas gdy zaawansowane metody jak kukulcze czy Robin Hood pozwalają utrzymywać stabilność operacyjną do wyższych wartości *load factor* bez utraty stałego czasu wyszukiwania.